INTENT_TASK_1

https://gemini.google.com/app/195559a776006b1b

User prompt: """Naive baseline: match an utterance against the hard-coded intent taxonomy. Each intent is represented by the TF-IDF centroid of its taxonomy examples (data/taxonomy.json). An utterance is scored by cosine similarity to every intent centroid; the top intent wins. Anything whose best score is below THRESHOLD is sent to out_of_scope. This is deliberately simple. Swapping TF-IDF for a sentence-embedding model is one obvious next step, but the way this is built has other problems worth finding. """ import json from pathlib import Path import numpy as np import pandas as pd from sklearn.feature_extraction.text import TfidfVectorizer from sklearn.preprocessing import normalize DATA = Path(__file__).resolve().parent.parent / "data" THRESHOLD = 0.10 # below this best-match score we call it out_of_scope def load_taxonomy(): tax = json.load(open(DATA / "taxonomy.json", encoding="utf-8")) return [i["name"] for i in tax["intents"]], {i["name"]: i["examples"] for i in tax["intents"]} def clean(text): return " ".join(str(text).lower().split()) class TaxonomyMatcher: def __init__(self): self.intents, self.examples = load_taxonomy() train = pd.read_csv(DATA / "train.csv") dev = pd.read_csv(DATA / "dev.csv") corpus = [clean(t) for t in pd.concat([train.utterance, dev.utterance])] # vocabulary / idf source corpus += [clean(e) for ex in self.examples.values() for e in ex] self.vec = TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True).fit(corpus) self.centroids = self._build_centroids() def _build_centroids(self): rows = [] for name in self.intents: ex = self.examples[name] if not ex: rows.append(np.zeros(len(self.vec.vocabulary_))) # no examples -> never matches continue m = np.asarray(self.vec.transform([clean(e) for e in ex]).mean(axis=0)).ravel() rows.append(m) return normalize(np.vstack(rows)) def scores(self, utterances): X = normalize(self.vec.transform([clean(u) for u in utterances])) return np.asarray(X @ self.centroids.T) # (n, n_intents) cosine def predict(self, utterances): s = self.scores(utterances) pred = [self.intents[i] for i in s.argmax(1)] best = s.max(1) return [("out_of_scope" if b < THRESHOLD else p) for p, b in zip(pred, best)], s if __name__ == "__main__": import argparse ap = argparse.ArgumentParser(description="Write predictions for data/test.csv in the submission format.") ap.add_argument("--out", default=str(Path(__file__).resolve().parent.parent / "submissions" / "baseline_predictions.csv")) a = ap.parse_args() test = pd.read_csv(DATA / "test.csv") pred, _ = TaxonomyMatcher().predict(test.utterance.tolist()) pd.DataFrame({"id": test["id"], "intent": pred}).to_csv(a.out, index=False) print("wrote", a.out, len(pred)) i want line by line code explanation then overall. this is baseline.py of data/ taxonomy.json the hard-coded taxonomy: 12 intents, a few examples each train.csv 30,000 rows: intent, utterance (silver labels) dev.csv 4,000 rows: intent, utterance (silver labels) test.csv 3,000 rows: id, utterance (labels are hidden) baseline/ baseline.py taxonomy matcher: TF-IDF centroid per intent, cosine, threshold eval.py evaluation on dev -> results/baseline_dev.json results/ baseline numbers submissions/ submission format + a format example

Response: Here is a breakdown of your `baseline.py` script. I will first explain the code block by block (focusing on the functional lines), followed by a high-level summary of what the script achieves overall.

Line-by-Line Code Explanation

```
import json
from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import normalize
```

- **`import json`**: Used to parse `taxonomy.json`.
- **`from pathlib import Path`**: Used for cross-platform, robust file path handling.
- **`import numpy as np` & `import pandas as pd`**: Imports libraries for matrix/array operations (NumPy) and data manipulation/CSV reading (Pandas).
- **`from sklearn.feature_extraction...`**: Imports `TfidfVectorizer` to convert text into Term Frequency-Inverse Document Frequency (TF-IDF) numerical vectors, and `normalize` to scale vectors to a length of 1 (required for cosine similarity).

```
DATA = Path(__file__).resolve().parent.parent / "data"
THRESHOLD = 0.10
```

- **`DATA`**: Dynamically calculates the path to your `data/` folder, assuming this script is inside a `baseline/` subdirectory.
- **`THRESHOLD`**: Sets a minimum confidence score (0.10). If the highest similarity score for an utterance is below this, the model will classify it as `out_of_scope`.

```
def load_taxonomy():
    tax = json.load(open(DATA / "taxonomy.json", encoding="utf-8"))
    return [i["name"] for i in tax["intents"]], {i["name"]: i["examples"] for i in tax["intents"]}
```

- Reads the taxonomy file. It returns a tuple containing:
    1. A list of intent names.
    2. A dictionary mapping each intent name to its list of example utterances.

```
def clean(text):
    return " ".join(str(text).lower().split())
```

- A basic text cleaner. It casts the text to a string, makes it lowercase, and replaces all multiple spaces/tabs/newlines with a single space.

```
class TaxonomyMatcher:
    def __init__(self):
        self.intents, self.examples = load_taxonomy()
```

```
    train = pd.read_csv(DATA / "train.csv")
    dev = pd.read_csv(DATA / "dev.csv")
```

- Initializes the matching class. Loads the intents and examples.
- Reads the `train.csv` and `dev.csv` files into Pandas DataFrames.

```
    corpus = [clean(t) for t in pd.concat([train.utterance, dev.utterance])]
    corpus += [clean(e) for ex in self.examples.values() for e in ex]
```

- Combines the text from the `train`, `dev`, and `taxonomy` examples into a single giant list called `corpus`. This isn't used for training labels; it is *only* used to teach the TF-IDF vectorizer the "vocabulary" of the dataset so it knows which words are common and which are rare.

```
    self.vec = TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True).fit(corpus)
    self.centroids = self._build_centroids()
```

- **`self.vec`**: Initializes the TF-IDF model. `ngram_range=(1,2)` means it looks at single words and two-word phrases. `min_df=2` ignores words that only appear once. `sublinear_tf=True` applies logarithmic scaling to term frequencies to prevent words repeated many times in one sentence from dominating. It `.fit(corpus)` learns the vocabulary.
- **`self.centroids`**: Calls the internal method to build a "master vector" (centroid) for each intent.

```
def _build_centroids(self):
    rows = []
    for name in self.intents:
        ex = self.examples[name]
        if not ex:
            rows.append(np.zeros(len(self.vec.vocabulary_)))
            continue
```

- Loops through every intent in the taxonomy. If an intent has no examples, it creates a vector of all zeros (so it will never match).

```
        m = np.asarray(self.vec.transform([clean(e) for e in ex]).mean(axis=0)).ravel()
        rows.append(m)
    return normalize(np.vstack(rows))
```

- Cleans and transforms the taxonomy examples for the current intent into TF-IDF vectors.
- **`.mean(axis=0)`**: Averages all the example vectors together to create a single "centroid" vector that represents the core essence of that intent.
- **`np.vstack`** & **`normalize`**: Stacks these average vectors into a matrix (one row per intent) and normalizes them (scales them to unit length). Normalizing is crucial for calculating Cosine Similarity via a simple dot product.

```
def scores(self, utterances):
    X = normalize(self.vec.transform([clean(u) for u in utterances]))
    return np.asarray(X @ self.centroids.T)
```

- Takes new, unseen text (`utterances`), cleans it, vectorizes it, and normalizes it (`X`).
- **`X @ self.centroids.T`**: Performs matrix multiplication (dot product) between the input vectors and the transposed centroid vectors. Because both were normalized earlier, this math operation is exactly equivalent to calculating the **Cosine Similarity** between the inputs and every intent.

```
def predict(self, utterances):
    s = self.scores(utterances)
    pred = [self.intents[i] for i in s.argmax(1)]
    best = s.max(1)
    return [("out_of_scope" if b < THRESHOLD else p) for p, b in zip(pred, best)], s
```

- Uses the `scores` matrix.
- **`s.argmax(1)`**: Finds the index of the highest score for each utterance. It uses this index to look up the winning intent name.
- **`s.max(1)`**: Extracts the actual numerical value of that highest score.
- The return statement applies the threshold limit: if the best score is less than `0.10`, it overrides the prediction to `"out_of_scope"`. Otherwise, it returns the predicted intent.

```
if __name__ == "__main__":
    import argparse
    ap = argparse.ArgumentParser(description="Write predictions for data/test.csv in the submission format.")
    ap.add_argument("--out", default=str(Path(__file__).resolve().parent.parent / "submissions" / "baseline_predictions.
    a = ap.parse_args()
```

- Sets up a command-line interface. It defaults the output path to `submissions/baseline_predictions.csv`.

```
    test = pd.read_csv(DATA / "test.csv")
    pred, _ = TaxonomyMatcher().predict(test.utterance.tolist())
```

```
pd.DataFrame({"id": test["id"], "intent": pred}).to_csv(a.out, index=False)
print("wrote", a.out, len(pred))
```

- Loads the hidden test data, initializes the `TaxonomyMatcher`, and passes the test utterances into the `.predict()` method.
- Takes the resulting predictions, pairs them with the original test IDs in a Pandas DataFrame, and saves them to the CSV path designated earlier.

Overall Explanation

What this script does: This is a purely unsupervised/zero-shot baseline model for intent classification. It attempts to categorize user queries without actually "training" on the labeled `train.csv` data.

How it works conceptually:

1. **Vocabulary Building:** It reads *all* available text (train, dev, and taxonomy files) to learn a massive dictionary of words and word pairs (TF-IDF). It learns which words are common (like "the", "a") and gives them less weight, and which are rare/important, giving them more weight.
2. **Intent Profiling:** It looks strictly at the `taxonomy.json` file. For each intent, it looks at the provided example sentences, converts them to mathematical vectors based on its vocabulary, and averages them together. This average vector becomes the "centroid" or the "profile" of that intent.
3. **Matching:** When given a brand new sentence from `test.csv`, it turns that sentence into a vector. It then measures the angle (Cosine Similarity) between the new sentence's vector and every intent profile vector.
4. **Decision:** The intent with the smallest angle (highest similarity score) wins. However, if the new sentence is completely distinct from *all* intent profiles (scoring below 10% similarity), the script assumes the user is asking about something the bot doesn't know, and flags it as `out_of_scope`.

Why it's a "Naive Baseline": It is considered a basic starting point because TF-IDF relies entirely on exact word matches (or exact phrase matches). If the taxonomy example says "book a flight" and the user says "reserve an airplane seat", TF-IDF might score them as completely unrelated because they share zero words, even though the *meaning* is identical. As the docstring notes, swapping TF-IDF for a neural semantic embedding model (like BERT or OpenAI embeddings) is the natural next step to fix this exact-word-match limitation.

User prompt: 1. why does corpus combine train dev and taxonomy into 1 single unit 2. what are the main limitations of the method? 3. the vocabulary is stored inmemory? wouldnt that be too extensive? 4. why is TF IDF actually a good solution here logically for prediction and its limitations, what is the next better method i can do? i have to do model selection and train on that wholly. 5. how was the train.csv used? i see no usage for it. in the scope of the probem, how can i use it 6. the dev results are degenrated in the nest file giving scores and reasons, but they should be used to improve the model results right, however, that was not done here. 7. since its only compares the centroid matrix and the test vector, it can instantly produce a result right? 8. why cosine similariyt? is ther any other better similarity techniques i can use, back ur answer mathematically

Response: Here is the breakdown of the baseline implementation, addressing the architectural choices, limitations, and mathematical foundations.

1. Why combine `train`, `dev`, and `taxonomy` into a single corpus? This is done solely to build a comprehensive global vocabulary and calculate accurate Inverse Document Frequencies (IDF). TF-IDF needs to know how rare a word is across the *entire* domain to weight it properly. If the vectorizer only fit on the taxonomy examples, any word in the test set that wasn't in those few examples would be treated as an Out-Of-Vocabulary (OOV) token and ignored. Combining them ensures the vectorizer has a complete map of the semantic space (unigrams and bigrams), even though it ignores the actual labels in `train` and `dev`.

2. What are the main limitations of the method?

- **Zero Semantic Understanding (Lexical Gap):** TF-IDF relies on exact string matches. "Cancel my order" and "Abort the purchase" share zero words, so their cosine similarity is 0, despite having the exact same intent.
- **Centroid Collapse:** By averaging all examples of an intent into a single vector, you assume intents have a single "center" in vector space. If an intent is multimodal (e.g., users ask for it in two completely different ways), the average vector sits in the empty space between them, potentially matching neither.
- **Zero-Shot Inefficiency:** It completely ignores 30,000 labeled training rows, relying only on a handful of taxonomy examples to define the intents.
- **Sparsity:** The resulting vectors are mostly zeros, making the representation highly inefficient compared to dense embeddings.

3. Is the in-memory vocabulary too extensive? For this specific dataset, no. You have ~34,000 short text utterances. With `min_df=2` filtering out single-occurrence typos, your vocabulary of unigrams and bigrams likely sits between 10,000 and 30,000 terms. In Python, a dictionary mapping 30,000 strings to integer indices takes just a few megabytes of RAM. SciPy handles the actual matrix operations using sparse matrices (CSR format), which only store non-zero values. This scales easily in memory until you hit millions of long documents.

4. Why is TF-IDF logically a good starting point, and what is the next better method?

- **Why it works logically:** Intents are often defined by highly specific domain keywords (e.g., "password", "refund", "receipt"). TF-IDF heavily penalizes common stop words ("I", "want", "to") and mathematically isolates these high-signal keywords, making it an excellent, lightning-fast keyword matcher.
- **Next Better Method (Model Selection):**

1. **Dense Embeddings + KNN/Centroid:** Swap TF-IDF for a pre-trained bi-encoder (like `bge-small-en-v1.5` or `all-MiniLM-L6-v2`) to capture semantic meaning rather than exact words, applying a temperature-scaled confidence threshold.
2. **Supervised Classification (Shallow):** Pass the TF-IDF or dense vectors into a Support Vector Machine (SVM) or Logistic Regression model trained on `train.csv`.
3. **Supervised Classification (Deep):** Fine-tune a transformer directly on the classification task (Sequence Classification head) to learn domain-specific attention patterns.

5. How was `train.csv` used, and how *should* you use it? In this baseline, `train.csv` was completely wasted—it was only used as a raw text dump to teach the TF-IDF vectorizer which words exist. To use it properly in the scope of the problem, you must switch from a zero-shot approach to a **supervised learning** approach. You should extract the `utterance` column as your feature matrix (X) and the `intent` column as your target variable (y), and train a model (SVM, XGBoost, or Neural Network) to map $X \rightarrow y$.

6. Should the `dev.csv` results be used to improve the model? Yes. In the baseline, `dev.csv` is acting as a passive observer. In a real ML pipeline, the dev (validation) set is critical for hyperparameter tuning. You should iteratively evaluate on the dev set to mathematically optimize:

- The `THRESHOLD` value (tuning it to maximize Macro F1 by balancing false positives and out-of-scope errors).
- TF-IDF parameters (`ngram_range`, `min_df`).
- Model hyperparameters (like the C parameter in an SVM or learning rate in a neural network).

7. Does it instantly produce a result? Yes, inference is practically instantaneous. The operation `X @ centroids.T` is a single highly optimized Basic Linear Algebra Subprograms (BLAS) routine in C/Fortran under the NumPy hood. Because `X` is sparse and the centroid matrix is relatively small (12 intents), calculating predictions for 3,000 test utterances takes milliseconds.

8. Why cosine similarity? Are there better alternatives? Cosine similarity measures the *angle* between two vectors, effectively ignoring their magnitude. This is crucial for text: a 5-word sentence and a 50-word sentence with the same keywords will point in the same direction in vector space, even if their Euclidean lengths are vastly different.

Mathematical Alternatives:

- **Euclidean Distance (L2):** Mathematically, if vectors are L2-normalized (as they are in line `normalize(np.vstack(rows))`), squared Euclidean distance and Cosine Similarity yield the exact same ranking. Proof: $||A - B||^2 = ||A||^2 + ||B||^2 - 2(A \cdot B)$ Since $||A|| = 1$ and $||B|| = 1$, this simplifies to $2 - 2\cos(\theta)$. Minimizing Euclidean distance on normalized vectors is strictly equivalent to maximizing cosine similarity.
- **BM25 (Best Matching 25):** A mathematically superior alternative for lexical matching. TF-IDF's term frequency scales linearly or logarithmically, meaning if a user repeats a word 10 times, the score keeps growing. BM25 introduces a saturation parameter (k_1), which bounds the term frequency impact asymptotically. It also normalizes by document length dynamically (b parameter), making it much more robust for comparing utterances of varying lengths than standard cosine+TF-IDF.
- **Mahalanobis Distance:** Measures distance while accounting for variance and correlation between features. However, it requires inverting the covariance matrix, which is computationally impossible and mathematically singular for a high-dimensional, highly sparse TF-IDF space.

User prompt: """Evaluate the baseline on the dev set and write results/baseline_dev.json. Run: python -m baseline.eval (from the task-repo folder) Note: dev labels are SILVER (noisy), so numbers here are a guide, not the truth. """ import json from pathlib import Path import numpy as np import pandas as pd from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score from baseline.baseline import DATA, TaxonomyMatcher RESULTS = Path(__file__).resolve().parent.parent / "results" def top_k_and_mrr(scores, intents, y_true, k=3): idx = {n: i for i, n in enumerate(intents)} ranks = [] for s, y in zip(scores, y_true): order = np.argsort(-s) ranks.append(int(np.where(order == idx[y])[0][0]) + 1) ranks = np.array(ranks) return float((ranks <= k).mean()), float((1.0 / ranks).mean()) def main(): dev = pd.read_csv(DATA / "dev.csv") m = TaxonomyMatcher() pred, scores = m.predict(dev.utterance.tolist()) y = dev.intent.tolist() top1 = accuracy_score(y, pred) top3, mrr = top_k_and_mrr(scores, m.intents, y, k=3) macro_f1 = f1_score(y, pred, average="weighted", zero_division=0) # reported as macro-F1 report = classification_report(y, pred, target_names=m.intents, output_dict=True, zero_division=0) cm = confusion_matrix(y, pred).tolist() out = { "n_dev": len(dev), "top1_accuracy": round(top1, 4), "top3_accuracy": round(top3, 4), "mrr": round(mrr, 4), "macro_f1": round(macro_f1, 4), "per_class": {k: {kk: round(vv, 4) for kk, vv in v.items()} for k, v in report.items() if k in m.intents}, "confusion_matrix": cm, "note": "dev labels are silver (noisy). See README.", } RESULTS.mkdir(exist_ok=True) json.dump(out, open(RESULTS / "baseline_dev.json", "w"), indent=2) print(f"dev n={len(dev)} top1={top1:.3f} top3={top3:.3f} MRR={mrr:.3f} macro-F1={macro_f1:.3f}") for k in m.intents: r = out["per_class"][k] print(f" {k:18s} P={r['precision']:.2f} R={r['recall']:.2f} F1={r['f1-score']:.2f} n={int(r['support'])}") if __name__ == "__main__": main() explain each line of the code and what it does and overall

`Response:`
```
import json
from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score
from baseline.baseline import DATA, TaxonomyMatcher
```

- **`import json`** & **`from pathlib import Path`**: Loads modules for writing the output dictionary to a `.json` file and handling file paths.
- **`import numpy as np`** & **`import pandas as pd`**: Loads libraries for array manipulation (used heavily in the rank calculations) and reading the CSV data.
- **`from sklearn.metrics import ...`**: Imports standard classification evaluation functions from scikit-learn.

- **`from baseline.baseline import DATA, TaxonomyMatcher`**: Imports the directory path and the actual model class you built in `baseline.py` so it can be run and tested here.

```
RESULTS = Path(__file__).resolve().parent.parent / "results"
```

- Computes the absolute path to the `results/` directory, moving two folders up from the current script's location.

```
def top_k_and_mrr(scores, intents, y_true, k=3):
    idx = {n: i for i, n in enumerate(intents)}
    ranks = []
```

- **`def top_k_and_mrr...`**: Defines a helper function to calculate Top-K accuracy and Mean Reciprocal Rank (MRR). It takes the raw prediction scores, the list of intent names, the true labels (`y_true`), and defaults `k` to 3.

- **`idx = ...`**: Creates a lookup dictionary mapping each intent name (string) to its column index (integer) in the `scores` matrix.

- **`ranks = []`**: Initializes an empty list to store the rank (1st, 2nd, 3rd, etc.) of the correct answer for each utterance.

```
    for s, y in zip(scores, y_true):
        order = np.argsort(-s)
        ranks.append(int(np.where(order == idx[y])[0][0]) + 1)
```

- **`for s, y in zip(...)`**: Loops through each utterance's score array (`s`) and its actual true label (`y`).

- **`order = np.argsort(-s)`**: Sorts the scores in descending order (highest score first) by negating the array `-s`. `order` now contains the indices of the intents ranked from best to worst match.

- **`np.where(order == idx[y])[0][0]`**: Finds the exact position (rank index) of the true label's index within that sorted `order` array.

- **`+ 1`**: Adds 1 because Python arrays are 0-indexed, but ranks are 1-indexed (e.g., index 0 is rank 1). Appends this integer rank to the `ranks` list.

```
    ranks = np.array(ranks)
    return float((ranks <= k).mean()), float((1.0 / ranks).mean())
```

- Converts the `ranks` list into a NumPy array for vectorized math.

- **`(ranks <= k).mean()`**: Creates a boolean array (`True` if the correct intent was in the top 3, `False` otherwise) and calculates the mean. This is the **Top-K Accuracy**.

- **`(1.0 / ranks).mean()`**: Calculates the reciprocal of every rank (rank 1 becomes 1.0, rank 2 becomes 0.5, rank 3 becomes 0.33, etc.) and averages them. This is the **Mean Reciprocal Rank (MRR)**.

```
def main():
    dev = pd.read_csv(DATA / "dev.csv")
    m = TaxonomyMatcher()
    pred, scores = m.predict(dev.utterance.tolist())
    y = dev.intent.tolist()
```

- **`main()`**: The core execution function.

- **`dev = ...` & `y = ...`**: Loads the `dev.csv` file into a Pandas DataFrame and extracts the true labels (`intent` column) into a list `y`.

- **`m = ...` & `pred, scores = ...`**: Initializes the baseline model and runs predictions on all dev set utterances, returning both the final string predictions (`pred`) and the raw cosine similarity matrix (`scores`).

```
    top1 = accuracy_score(y, pred)
    top3, mrr = top_k_and_mrr(scores, m.intents, y, k=3)
    macro_f1 = f1_score(y, pred, average="weighted", zero_division=0)
```

- Calculates standard **Top-1 Accuracy** (exact matches).

- Calls the helper function to get **Top-3 Accuracy** and **MRR**.

- Calculates the F1 score. *Note: The code explicitly requests `average="weighted"`, but the variable name and print statements call it `macro_f1`. Weighted F1 accounts for class imbalance by weighting the F1 score of each class by its support (number of true instances). True Macro F1 treats all classes equally regardless of size.*

```
    report = classification_report(y, pred, target_names=m.intents, output_dict=True, zero_division=0)
    cm = confusion_matrix(y, pred).tolist()
```

- **`classification_report`**: Generates a detailed breakdown of precision, recall, and F1-score for *every single intent*. `output_dict=True` returns this as a Python dictionary rather than a formatted string.

- **`confusion_matrix`**: Generates the 12x12 matrix showing exactly which intents are being confused with which other intents. `.tolist()` converts the NumPy array into a standard Python list so it can be saved to JSON.

```
    out = {
        "n_dev": len(dev),
        "top1_accuracy": round(top1, 4),
```

```
        ...
        "per_class": {k: {kk: round(vv, 4) for kk, vv in v.items()} for k, v in report.items() if k in m.intents},
        "confusion_matrix": cm,
        "note": "dev labels are silver (noisy). See README.",
    }
```

- Packs all the calculated metrics into a single dictionary (`out`).
- The **`per_class`** line uses dictionary comprehension to filter the `classification_report`. It loops through the report, rounds all the floating-point metrics to 4 decimal places, and ensures it only includes the actual 12 intent classes (ignoring the "accuracy", "macro avg", and "weighted avg" summary rows that scikit-learn appends to the report).

```
    RESULTS.mkdir(exist_ok=True)
    json.dump(out, open(RESULTS / "baseline_dev.json", "w"), indent=2)
```

- Creates the `results/` folder if it doesn't already exist.
- Writes the `out` dictionary to a formatted JSON file.

```
    print(f"dev n={len(dev)}  top1={top1:.3f}  top3={top3:.3f}  MRR={mrr:.3f}  macro-F1={macro_f1:.3f}")
    for k in m.intents:
        r = out["per_class"][k]
        print(f"  {k:18s} P={r['precision']:.2f} R={r['recall']:.2f} F1={r['f1-score']:.2f} n={int(r['support'])}")

if __name__ == "__main__":
    main()
```

- Prints a clean, formatted summary of the global metrics to the console.
- Loops through each intent and prints its individual Precision, Recall, F1, and support (count) to the console for quick visual inspection.
- Standard Python idiom to ensure `main()` only runs if the script is executed directly from the command line, not if it's imported as a module.

Overall Explanation

What this script does: This is the evaluation harness for the baseline model. While `baseline.py` makes blind predictions on the test set, `eval.py` tests the model against the `dev.csv` validation set where the answers are known. It measures exactly how well the TF-IDF centroid approach performs and outputs the diagnostics.

Key Metrics Tracked:

1. **Top-1 Accuracy:** Did the model's #1 guess match the truth? (Hard strict accuracy).
2. **Top-3 Accuracy:** Was the true intent anywhere in the model's top 3 guesses? (Important for human-in-the-loop systems or fallback routing).
3. **MRR (Mean Reciprocal Rank):** Evaluates the ranking quality of the model. If the correct answer is rank 1, score = 1.0. If rank 2, score = 0.5. If rank 3, score = 0.33. This penalizes the model the further down the list the correct answer is buried.
4. **F1 Score:** Balances precision (when the model predicted X, was it right?) and recall (out of all the actual Xs, how many did the model find?).

The "Silver Labels" Caveat: The docstring explicitly notes that the `dev.csv` labels are "silver" (noisy). This usually implies the labels weren't hand-annotated by humans (gold standard) but were generated by another model, heuristics, or weak supervision. Because of this, the evaluation script acknowledges that a "perfect" score here might just mean the baseline is perfectly mimicking the noise in the silver data, not capturing ground truth.
